MLOPS CAPSTONE · PROJECT 4

From Raw Listings to a Monitored Model

An end-to-end MLflow pipeline that predicts house prices, refuses bad data and bad models, and keeps the previous model serving when anything fails.

Group 4 — Swagata Bhowmik (B053) · Sumit Kothari (B062)

Elective II: Machine Learning Operations · Dr. Santosh Kumar · 24 October 2026

±₹5.74 L	₹5.78 L	8 · 2	3 × 500	0 errors	16 / 16
average error (validation)	held-out test error	pipeline stages · safety gates	incoming listings monitored	in 300 live requests	tests on Linux & Windows

Executive summary

A real-estate start-up prices new listings by hand, with no record of which data or code produced a model and nothing to stop a bad release. We built one automated pipeline that takes the supplied data to a live, monitored model.

Every run is tracked in **MLflow**; a **data gate** and a **model gate** stop bad inputs and weak models; every change is **tested, released and deployed by CI/CD**; and three later batches are **monitored for drift**. A live dashboard lets anyone try the model and trace it back to its code and data.

The chosen Ridge model is off by ₹5.74 lakh on average — about 9% of a typical ₹62-lakh listing — against ₹17.73 lakh for a naive guess. All eight requested deliverables are complete and evidenced.

Live dashboard	Code repository	API documentation
group4-mlflow-capstone.onrender.com/	github.com/Swagata-Bhowmik/group4-mlflow-capstone	group4-mlflow-capstone.onrender.com/docs

Inside this report

	Section	What you will find
1	What we were asked to do	The brief, the eight deliverables with their status, and the success targets vs what we achieved
2	How we built it	The architecture, its eight numbered steps mapped to the MLOps lifecycle, and our technology choices
3	What we delivered	Each deliverable: what was asked, what we delivered, where to find it, with screenshots and evidence
4	Team and conclusion	Who did what, and the overall result

1 What we were asked to do

Project 4: Capstone — End-to-End MLflow Pipeline from Raw Data to Monitored Deployment

The brief asked for one automated pipeline that takes raw listings to a live, monitored model and fails safely: when data or model quality is not acceptable, it must stop cleanly and keep the previous model in service. It had to use free tools, automate every stage, let only the pipeline change the production model, and keep secrets out of the repository.

Deliverables and status

Deliverable	Status	Main evidence
Repository: pipeline, configuration, tests, workflows, Dockerfile, README	✓ Done	GitHub repository · section 3.1
Design note (max 2 pages) with architecture diagram	✓ Done	design_note.pdf · section 3.2
Workflow graph, run history and registry screenshots	✓ Done	output/screenshots · section 3.3
Evidence of two failing-gate scenarios	✓ Done	live_safe_failures.json, PR #2 · section 3.4
Lineage demonstration (live)	✓ Done	Dashboard Lineage tab, /lineage · section 3.5
CI history including at least one release	✓ Done	GitHub Actions, Releases · section 3.6
Monitoring dashboard or report for the three batches	✓ Done	Dashboard Monitoring tab, monitoring.html · section 3.7
Reflection (one page)	✓ Done	reflection.pdf · section 3.8

Problem → solution → how

The problem

- Retraining and releases are manual
- No record of data, code or settings behind a model
- Nothing stops bad data or a worse model
- No warning when new listings change

Our solution

- One automated 8-stage pipeline
- Two quality gates; old model keeps serving
- Full tracking and lineage in MLflow
- Tested, released and deployed by CI/CD

How it works

- Git + one validated config.yaml
- PR checks run tests and the model gate
- Merge → image tagged by commit → release → deploy
- Monitoring alerts start a new gated change

Success targets and what we achieved

Measure	Target (config.yaml)	Achieved	Result
Average error (MAE)	≤ ₹7.0 lakh	₹5.74 lakh	✓ Met
Large-error measure (RMSE)	≤ ₹9.0 lakh	₹7.52 lakh	✓ Met
New model vs live model	≤ 2% worse, or rejected	Enforced at evaluate	✓ Enforced
Live request errors	≤ 1%	0 of 300	✓ Met
p95 latency	≤ 200 ms	213 ms (1 user) · 857 ms (5 users)	⚠ Not met on free host
Reproducibility	MAE within 1e-6	Identical on a fresh clone	✓ Met

2 How we built it

Architecture mapped to the MLOps lifecycle

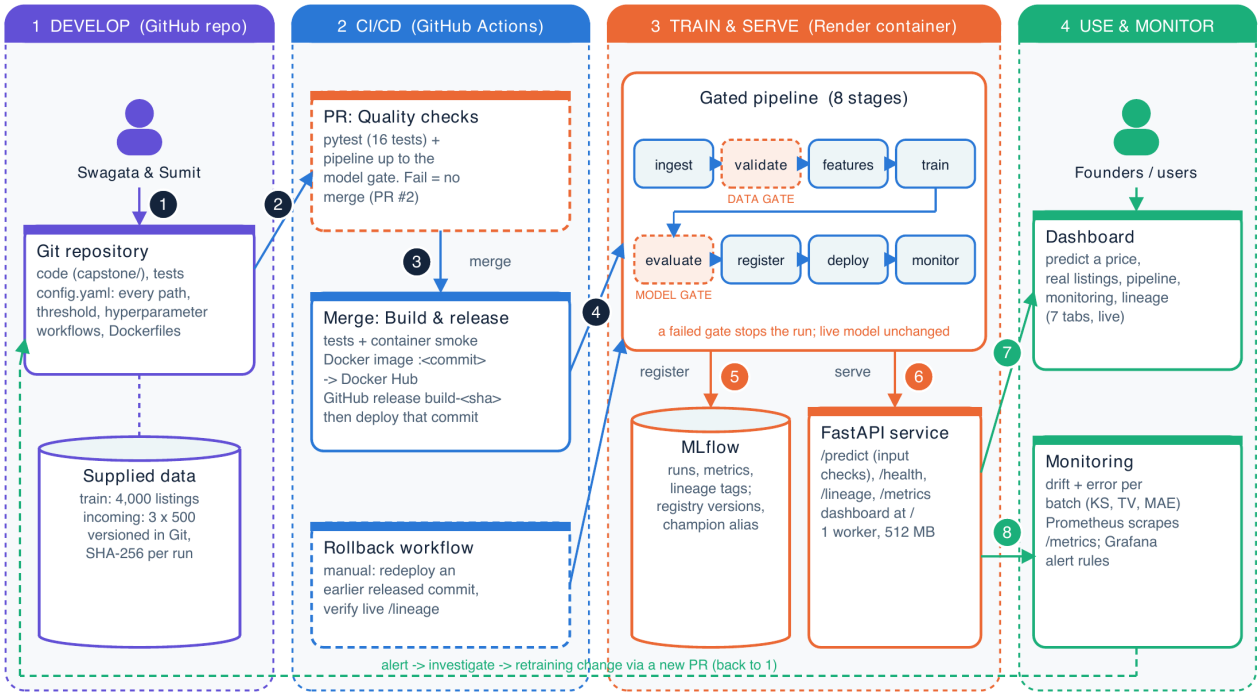


Figure 1 — Architecture: develop → CI/CD → train & serve → use & monitor. Numbers follow the steps below.

#	What happens	Lifecycle stage
1	Code, config.yaml (validated on load), tests and the supplied data are versioned together in Git	Framing, data & configuration
2	Every pull request runs the tests and the pipeline up to the model gate; a failure blocks the merge	Continuous integration
3	A merge builds and tests a container, tags it with the commit, pushes it to Docker Hub and creates a release	Continuous delivery
4	That exact commit is deployed to Render; the build runs the gated pipeline; rollback redeploys an earlier release	Deployment & orchestration
5–6	Validate and evaluate are gates; passing models are registered in MLflow and served by the API	Validation, tracking, serving
7–8	Founders use the live dashboard; batches are monitored, and an alert leads back to step 1	Consumption, monitoring & feedback

Technology choices

Need	What we used and why
Orchestration	A small persisted DAG in capstone/core.py — explicit stages, artifact hashes and reruns without an extra service (instead of Prefect/ZenML)
Tracking & registry	MLflow 3: runs, metrics, lineage tags, registered versions with lifecycle tags and a champion alias
Serving	FastAPI with strict input checks, Prometheus metrics and a 7-tab dashboard
CI/CD & packaging	GitHub Actions, Docker, Docker Hub, GitHub releases, automatic deployment to Render (free)
Data versioning & drift	Git + SHA-256 snapshots for the small supplied CSVs (instead of DVC); SciPy effect sizes for drift (instead of Evidently)

3 What we delivered

Each deliverable: what was asked, what we delivered, where to find it

3.1 Repository

What was asked	One repository with the pipeline, configuration, tests, workflows, Dockerfile and README.
What we delivered	• **Pipeline:** capstone/core.py — 8 stages with dependencies, hashes, logs, gates, promotion and rollback • **Configuration:** one config.yaml, validated on load (no hard-coded paths, thresholds or hyperparameters) • **Tests:** 16 automated tests, passing on Linux and Windows on every pull request • **Workflows:** PR quality checks, build & release, rollback • **Dockerfiles and README:** Docker Compose stack, Render image, and a README with setup for Windows, macOS and Linux
Where to find it	github.com/Swagata-Bhowmik/group4-mlflow-capstone

3.2 Design note with architecture diagram

What was asked	A design note of at most two pages with an architecture diagram, numeric and justified success thresholds, and every component traced to a lifecycle stage.
What we delivered	• Exactly 2 pages: objective, success metrics with plain-language meaning, architecture diagram with 8 numbered steps, lifecycle mapping • Gates, tracking and lineage, CI/CD and rollback, monitoring and alert response, tool choices and limits • Every number is generated from the recorded evidence files, so the note cannot drift from the results
Where to find it	output/pdf/design_note.pdf

3.3 Workflow graph, run history and registry screenshots

What was asked	Screenshots of the workflow graph, the run history and the model registry.
What we delivered	• Workflow graph of the live run, with both gates and the model-gate checks (dashboard Pipeline tab) • MLflow run history showing passed and failed runs with their metrics • MLflow registry: version 1 production with the champion alias, version 2 archived
Where to find it	output/screenshots/ (01, 02, 03, 04) · presentation slide 4

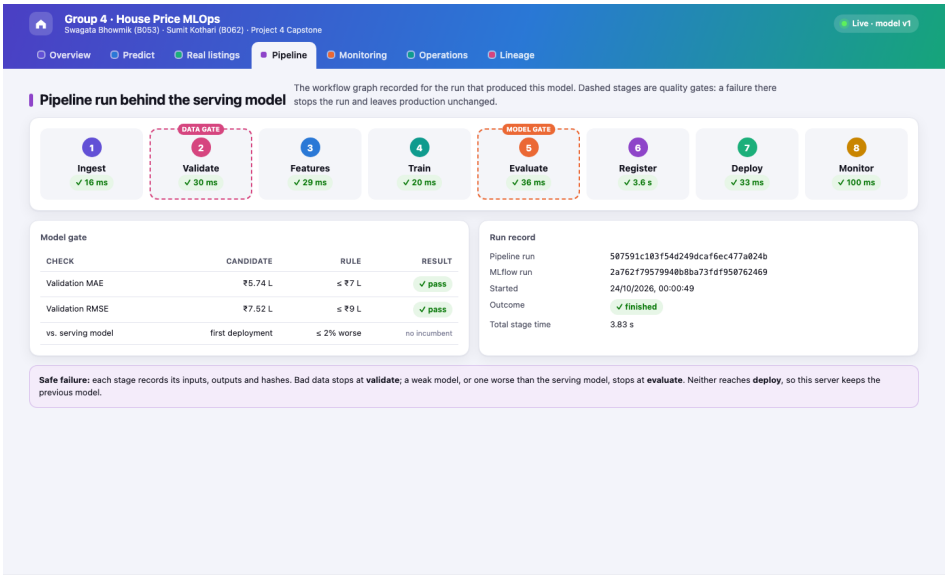


Figure 2 — Workflow graph of the run behind the live model (8 stages, data gate and model gate).

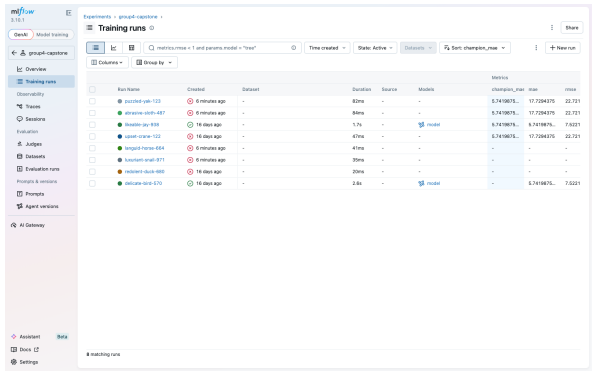


Figure 3 — MLflow run history

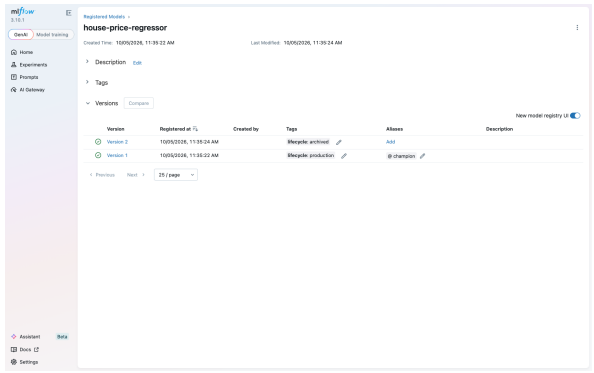


Figure 4 — MLflow model registry

3.4 Evidence of two failing-gate scenarios

What was asked	At least two failing scenarios — a bad data batch and a model below the threshold — with the production model and endpoint unchanged after each.
What we delivered	• Ran both against a real HTTP server while it kept answering prediction requests • Also showed a bad change blocked in CI (PR #2) and invalid input rejected by the live API
Where to find it	reports/live_safe_failures.json · scripts/live_safe_failure.py · PR #2

Scenario	Stopped at	Gate message	Live service during the run
Bad data: 20% of Age_Years blanked (limit 8%)	✕ validate (data gate)	Age_Years: missing fraction exceeds limit	56 requests · 0 errors · v1 unchanged
Bad model: median-only dummy, error ₹17.73 L	✕ evaluate (model gate)	Candidate exceeds absolute error thresholds	59 requests · 0 errors · v1 unchanged
Bad code change in PR #2: Ridge α = 1000, error ₹8.77 L	✕ CI quality checks	Model gate rejected the candidate	Not merged until fixed (α = 10)
Invalid input: 0 bedrooms	✕ API input checks (HTTP 422)	Bedrooms: outside allowed range	Model never called; unchanged

Why this matters

A failed gate stops the run before register and deploy, so the serving pointer and registry are never touched. The previous model keeps answering throughout — 115 live requests with zero errors across both failures.

3.5 Lineage demonstration (live)

What was asked	A live lineage demonstration starting from the deployed model; run records must include data version and configuration.
What we delivered	• From the live model: Git commit → MLflow run → training-data SHA-256 → full configuration → model file hash • Shown live on the dashboard Lineage tab and the public /lineage endpoint; the same tags are recorded on every MLflow run • Registry changes (production, archived, champion) happen only through the pipeline's deploy and rollback functions
Where to find it	group4-mlflow-capstone.onrender.com/lineage · output/screenshots/05, 04

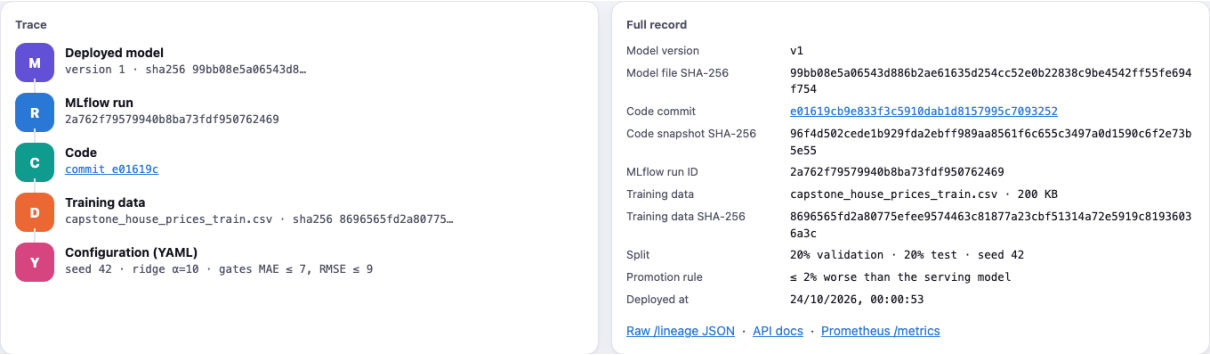


Figure 5 — Live lineage of the deployed model: trace (left) and full record (right).

3.6 CI history including a release

What was asked	A pull request must trigger tests and a lightweight pipeline run; a merge to main must build a deployable image and release it; CI history must show a passing and a failing pull request and at least one release; rollback must be documented and tested once.
What we delivered	• Quality checks on every PR, on Ubuntu and Windows: tests plus the pipeline up to the model gate • PR #2 failed on purpose (the model gate rejected α = 1000) and passed after the fix. CI also caught two real bugs: a test-collection error in PR #3 (fixed in the same PR) and a Windows text-encoding error in PR #8 (fixed in PR #9) • Every merge: tests → container smoke test → Docker image tagged with the commit → GitHub release build-<sha> → deploy that commit to Render → verify /lineage • Rollback workflow tested live: back to an earlier release and forward again, about 2 minutes each, both verified
Where to find it	GitHub Actions · Releases · reports/render_rollback.json · docs/OPERATIONS.md

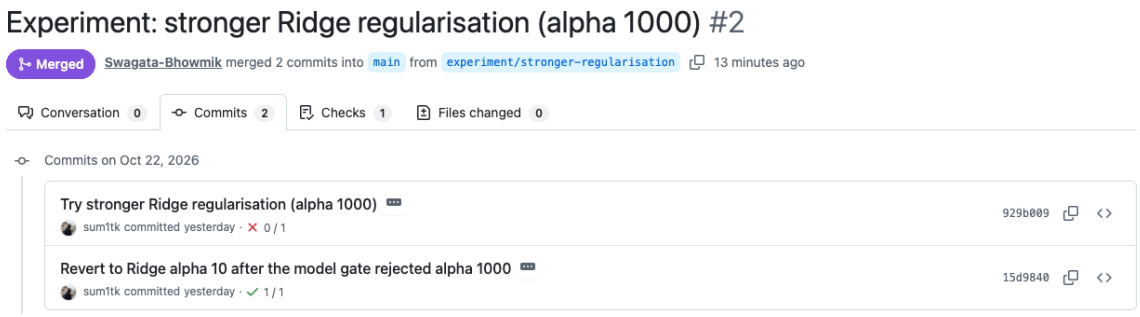


Figure 6 — PR #2: the α = 1000 commit failed CI (✕ 0/1), the fix passed (✓ 1/1).

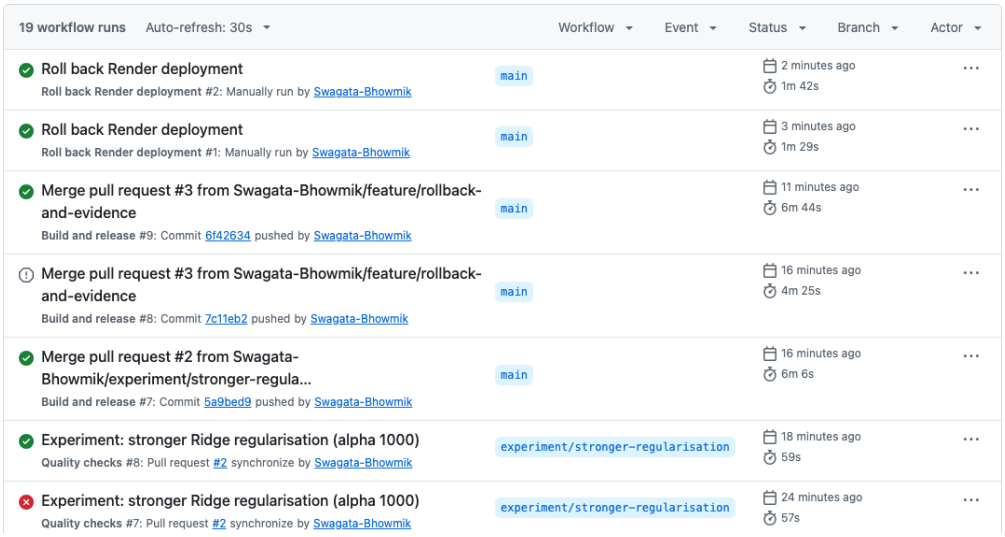


Figure 7 — GitHub Actions history: quality checks, build & release runs, and both rollback runs.

3.7 Monitoring of the three incoming batches

What was asked	A dashboard or report showing operational metrics and drift over the three batches, justified alert thresholds, the batches that trigger them, and what the pipeline should do when an alert fires.
What we delivered	• Each batch compared with the training data (data drift) and scored against its real prices (performance drift) • Alerts: numeric KS D or categorical total variation above 0.10; MAE above ₹7 L or 1.25 × validation error • Operational metrics live on /metrics and the Operations tab; Prometheus rules and a Grafana dashboard for a Docker host
Where to find it	Dashboard Monitoring tab · reports/monitoring.html · live dashboard

Batch	Error (MAE)	Status	What changed
Batch 1	₹5.56 L	✓ Stable	Nothing above threshold
Batch 2	₹6.87 L	⚠ Data drift	Houses got smaller (area); error still under the limit
Batch 3	₹13.11 L	✕ Performance alert	Area, bedrooms, bathrooms and city zone shifted; error doubled

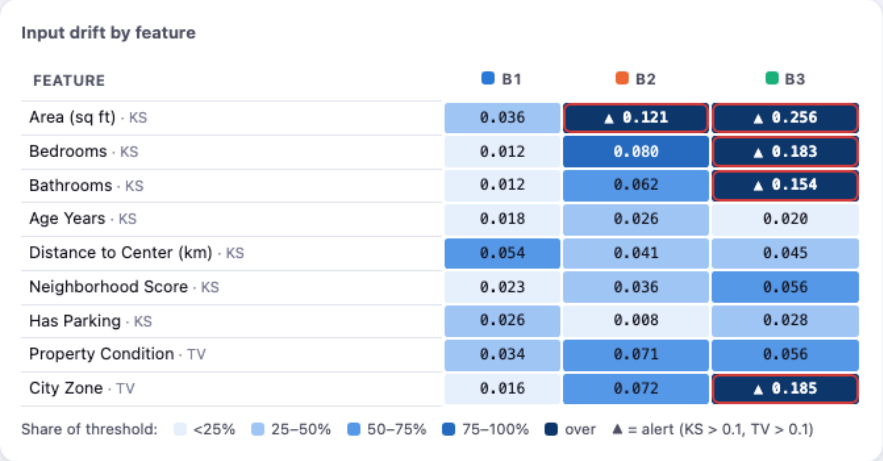


Figure 8 — Drift per feature; outlined cells are above the 0.10 alert threshold.

When an alert fires

Nothing is replaced automatically. The live model keeps serving while the team investigates and starts a new pipeline run with fresh labelled data; a retrained model goes live only if it passes the data gate, the error limits and the comparison with the live model.

300	0	213 ms	857 ms
live requests (load test)	errors	p95, 1 user	p95, 5 users (target 200)

3.8 Reflection: what we would improve

What was asked	A short reflection (one page) on what we would improve with more time.
What we delivered	• **What worked:** every release decision is automatic, explainable and traceable • **More serving power:** more CPU and workers to meet the 200 ms target under load • **Persistent model store:** compare every cloud release with the live model • **Safer releases:** canary or blue/green deployment with automatic rollback • **Smarter alerts:** calibrated thresholds, per-zone errors and notifications • **Scale the tooling:** DVC for data and Prefect/ZenML for scheduling as the project grows
Where to find it	output/pdf/reflection.pdf

4 Team contributions and conclusion

Swagata Bhowmik (B053)	Sumit Kothari (B062)
• Pipeline core, gates, promotion and rollback • Serving API, monitoring and the live dashboard • Docker, release workflow and automatic Render deployment • Rollback workflow and live evidence (failing gates, load test, fresh clone) • Screenshots, presentation and Windows support	• Model comparison that selected Ridge (α = 10) • Test suite and measured evidence • Design note, reflection, demo and operations documents • CI gate experiment (PR #2): α = 1000 rejected by the model gate, then fixed

Based on the commit history of the repository.

Conclusion

All eight deliverables are complete and evidenced. The system turns the supplied data into a live, monitored model through eight automated, re-runnable stages; two gates stop bad data and weak models while the previous model keeps serving; every model is traceable to its code, data and configuration; and every change is tested, released and deployed by CI/CD, with rollback tested on the live service.

The main open limit is serving speed on the free host under concurrent load, which is the first improvement we would make.